C PROGRAMMING – ASSIGNMENT 2 (CO2)

Question 7: Day-4 Design Challenge: Diagnose, Redesign and Validate

Student name	P.SAI SUSHANTH REDDY
Register number	1925260171
GitHub repository	CSA02-<RegisterNumber>-Assignment2 (folder: Q7)
Link	https://github.com/nagagowthamjanjanam-code/CSA02-1965260183-Assignment-2
Compiler	gcc (Ubuntu 13.3.0-6ubuntu2~24.04.1) 13.3.0
Compile command	`gcc -Wall -Wextra -O2 -o q7_redesign q7_redesign.c`
Marks / Bloom level	13 marks / K5 – Evaluate

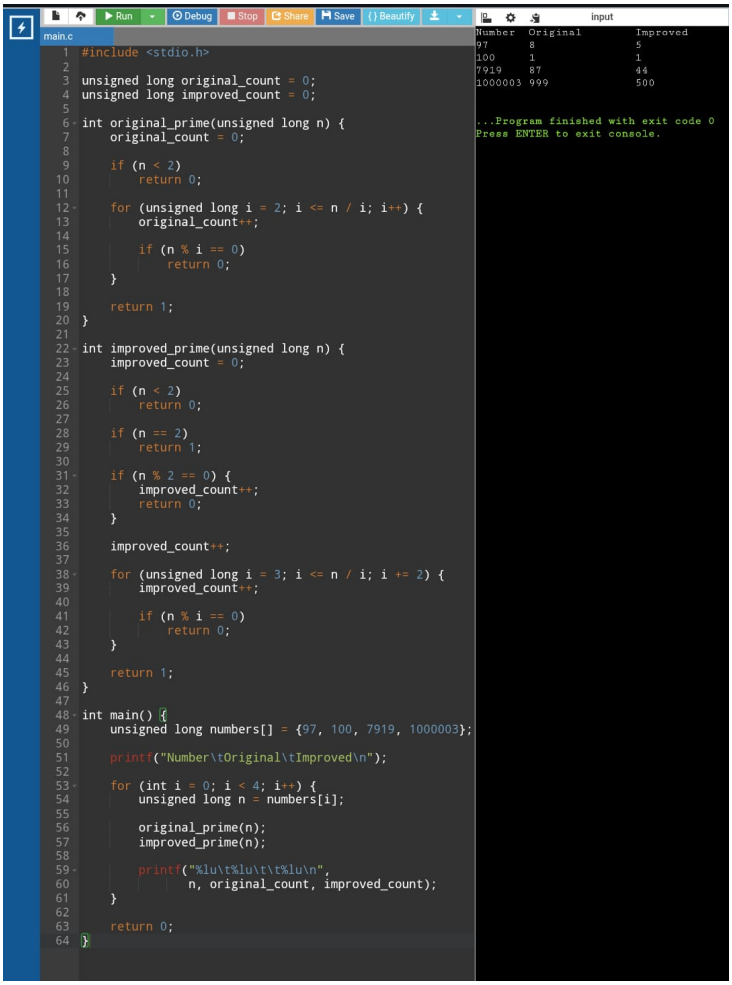
Selected program

Program chosen: Q5 – prime test for the lucky-draw tickets (trial division with the loop condition `i * i <= n`, called the *original* below). It is the program from Q1–Q6 whose cost visibly grows with the input: 999 divisions for the ticket 1000003, against 8 for 97. The change is **not** a switch between for, while and do-while – Q5 showed that the loop form leaves the division count unchanged. The redesign exploits a number-theoretic property of the divisors that actually need testing.

A. Diagnose the bottleneck (2 marks)

All figures come from the Q5 programs (appendix of this report, q7_redesign.c) over the tickets 2 … 1,000,003.

Question	Evidence from the original program
Where are most operations performed?	78,499 prime tickets (7.8% of the numbers) cause **50,659,938 of the 67,741,504 divisions (74.8%)**. A prime must survive every divisor up to √n; a composite stops at its smallest factor. Even tickets cost 1 division.
Which count grows with the input?	The number of divisions for a prime is ⌊√n⌋ − 1: **8 (n = 97), 87 (7919), 998 (999983), 999 (1000003)**. The Q5 screenshot (Figure 1) shows the same 8 / 1 / 87 / 999. It grows with √n: ten times more digits of work for every 100× larger ticket.
Is work unnecessarily repeated?	Yes. Of the 66,820,001 divisions that did not find a divisor, **33,640,395 (50.3%) used an even i** and **11,220,986 (16.8%) used an odd multiple of 3**: **67.1%** of all divisions test a divisor that cannot divide n once 2 and 3 have been tried (an even i dividing n would mean 2 divides n). Every ticket also re-tests 2, 3, 4, 5 … from scratch, sharing nothing with the previous ticket.




```c
#include <stdio.h>

unsigned long original_count = 0;
unsigned long improved_count = 0;

int original_prime(unsigned long n) {
    original_count = 0;

    if (n < 2)
        return 0;

    for (unsigned long i = 2; i <= n / i; i++) {
        original_count++;

        if (n % i == 0)
            return 0;
    }

    return 1;
}

int improved_prime(unsigned long n) {
    improved_count = 0;

    if (n < 2)
        return 0;

    if (n == 2)
        return 1;

    if (n % 2 == 0) {
        improved_count++;
        return 0;
    }

    improved_count++;

    for (unsigned long i = 3; i <= n / i; i += 2) {
        improved_count++;

        if (n % i == 0)
            return 0;
    }

    return 1;
}

int main() {
    unsigned long numbers[] = {97, 100, 7919, 1000003};

    printf("Number\tOriginal\tImproved\n");

    for (int i = 0; i < 4; i++) {
        unsigned long n = numbers[i];

        original_prime(n);
        improved_prime(n);

        printf("%lu\t%lu\t\t%lu\n",
               n, original_count, improved_count);
    }

    return 0;
}
```

Figure 1: Q5 comparison program output. 'Original' column = the program being diagnosed (8, 1, 87, 999); 'Improved' column = odd divisors only (5, 1, 44, 500), the first step of the redesign.

B. Design objective (2 marks)

The objective is measurable and derived from the diagnosis (67% of divisions are redundant):

#	Objective	How it is measured
O1	Reduce divisions for the worst-case ticket 1000003 from 999 to **≤ 350** (at least 65% fewer)	division counter (same counter as Q5)
O2	Reduce the **total** divisions over all tickets 2 … 1,000,003 by **≥ 60%** (67.74 M → ≤ 27.1 M)	sum of the counter over the range
O3	**Identical** prime / not-prime answer for every ticket	compare with the original for all 1,000,002 values
O4	**No extra memory**: O(1) space	no arrays
O5	Never noticeably worse than the original	compare the counter value for every n

C. Design of the revised solution (4 marks)

C.1 What changes and which property is used

Every integer is 6k, 6k+1, 6k+2, 6k+3, 6k+4 or 6k+5. The forms 6k, 6k+2 and 6k+4 are even and 6k+3 is divisible by 3. Hence, **once n has been tested for 2 and 3, any prime factor of n must be of the form 6k−1 or 6k+1** (5, 7, 11, 13, 17, 19, 23, 25 …). The smallest factor of a composite n is itself prime, so it is always among these candidates. The redesign therefore (1) tests 2 and 3 once, (2) tests only i = 6k−1 and i+2 = 6k+1 in each pass and advances i by 6, (3) keeps the early return and the condition `i * i <= n`. Only 2 of every 6 integers are tried (one third). This is a change of *which divisors are tested*, not of the loop statement.

Stages. R1 = test 2, then odd divisors only (half of the candidates; already visible in the student's Q5 screenshot). R2 = 6k±1 (one third) = the final redesign.

C.2 Pseudocode

```
is_prime_6k(n):
    checks = 0
    if n < 2 : return 0
    if n < 4 : return 1                  # 2 and 3 are prime
    checks++ ; if n mod 2 == 0 : return 0
    checks++ ; if n mod 3 == 0 : return 0
    i = 5
    while i * i <= n:
        checks++ ; if n mod i       == 0 : return 0     # candidate 6k-1
        checks++ ; if n mod (i + 2) == 0 : return 0     # candidate 6k+1
        i = i + 6
    return 1
```

Flowchart - Q7 redesigned test (6k +/- 1)

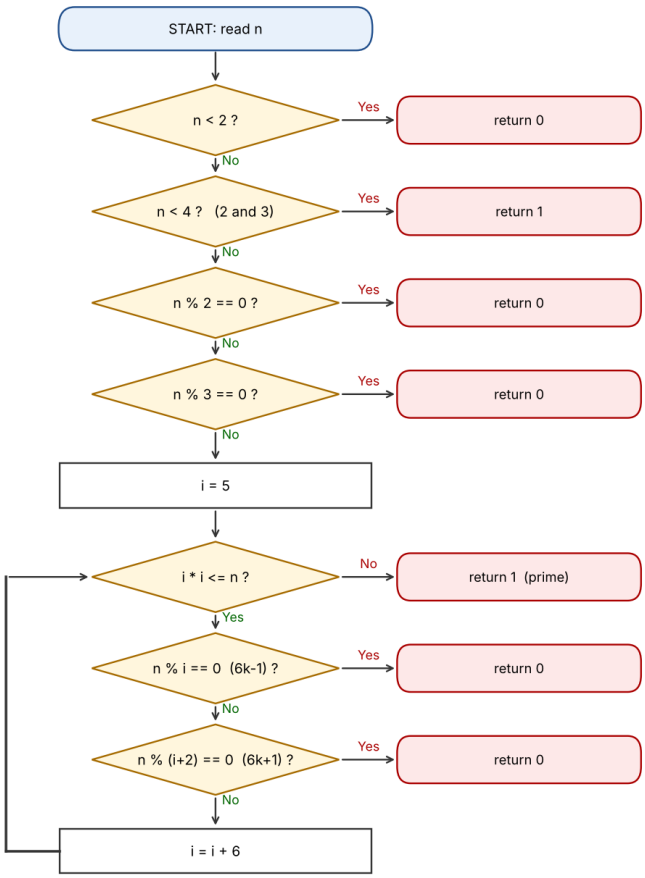


Figure 2: flowchart of the redesigned test

C.3 Extra memory and trade-offs

- **Extra memory:** none (two more comparisons, no arrays): O(1) space, same as the original.
- **Cost of the idea:** two divisions per pass share one loop test, so the test `i*i <= n` is made for i but the second candidate i+2 may lie just above √n. This is harmless (a divisor found there is still a genuine divisor of n, and i+2 = n is impossible for n ≥ 5) but can cost one division beyond the original's bound.
- **Small inputs:** 2 and 3 are always tested, so for n = 5 and 7 the redesign uses 2 divisions while the original uses 1.
- **Complexity of the code:** 10 lines of pseudocode instead of 4; an off-by-two error (i += 6 versus i += 2) would silently break results, so validation against the original is part of the design.

- **Still O(√n):** the improvement is a constant factor (about 3), not a new complexity class; a sieve (Q5) changes the class when many tickets are checked, at the price of O(N) memory.

D. Prediction before implementing (1 mark)

Measure	Prediction (reasoned before running)	Basis
Divisions for n = 1000003	**334** (original 999; R1 predicted 500)	2 and 3, plus the numbers ≡ ±1 (mod 6) in [5, 1000]: 332 values (333 in [1,1000] minus the number 1). 332 + 2 = 334. R1: 1 + 499 odd values from 3 to 999 = 500.
Divisions for 7919	≈ 30	√7919 ≈ 88.99: 2, 3 + 28 values ±1 mod 6 in [5, 88]
Total, all tickets 2 … 1,000,003	**55 – 66% fewer**	primes (75% of the work) improve by about 2/3; composites that have factor 2 or 3 (about 72% of them) exit at the same step in both versions, so their share improves less
Running time	falls by about the same percentage as the divisions (≈ 60 – 65%)	the 64-bit remainder dominates the loop; the extra comparison is cheap
Correctness	0 mismatches	every prime factor candidate is retained

E. Implement and validate (2 marks)

Both versions (and R1) are in q7_redesign.c; they were run on the **same inputs** with the same division counter, compiled with gcc 13.3.0 `-Wall -Wextra -O2`.

E.1 Same inputs, division counts (original → redesigned)

n	Result	Original	R1 odd only	R2 6k±1	R2 / original
97	prime	8	5	4	0.50
100	composite	1	1	1	1.00
7919	prime	87	44	30	0.34
999983	prime	998	500	334	0.33
1000001	composite	100	51	35	0.35
1000003	prime	999	500	**334**	0.33
2, 3	prime	0	0	0	–
25, 35	composite	4	3	3	0.75
49	composite	6	4	4	0.67

E.2 Whole range, n = 2 … 1,000,003 (1,000,002 numbers, 78,499 primes)

Version	Total divisions	Primes only	Composites only	Worst single n	Time (best of 5)
Original	67,741,504	50,659,938	17,081,566	999	0.2442 s
R1 odd only	34,601,109 (−48.9%)	25,388,823	9,212,286	500	0.1268 s
R2 6k±1	**23,739,568 (−65.0%)**	17,030,426 (−66.4%)	6,709,142 (−60.7%)	**334**	**0.0852 s** (−65%)

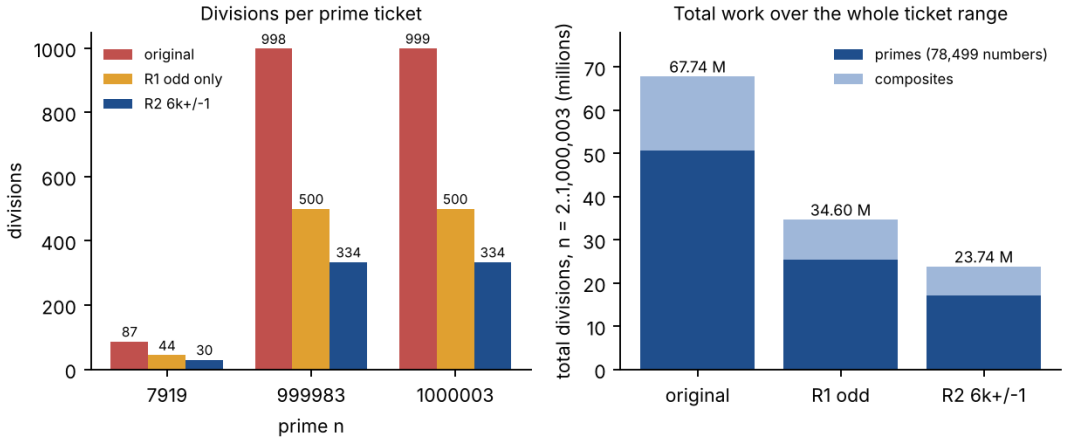


Figure 3: divisions per prime ticket and total work over the whole range (measured)

Validation. Results of the original, R1 and R2 were compared for every n from 2 to 1,000,003: **0 mismatches**, 78,499 primes in each case. Repeating the single check of 1000003 twenty thousand times took 0.0734 s (original), 0.0376 s (R1) and 0.0240 s (R2). A per-number comparison of the counters (q7_worse_check.c) gave:

```
R2 better on 333328 values, equal on 666672, worse on 2 (max extra divisions 1)
worse at n = 5 7
```

F. Defend the final design (2 marks)

Objective	Target	Measured	Met?
O1 divisions for 1000003	≤ 350	334 (999 → 334, −66.6%)	yes
O2 total work, 2 … 1,000,003	≥ 60% fewer	67,741,504 → 23,739,568 (−65.0%)	yes
O3 identical answers	0 mismatches	0 mismatches in 1,000,002 numbers	yes
O4 extra memory	none	none, O(1)	yes
O5 never noticeably worse	≤ original (+ small)	better on 333,328 numbers, equal on 666,672, worse on only 2 (n = 5 and 7) by 1 division	yes, with a stated exception

Prediction versus result. The division count for 1000003 (334) and for the per-ticket ratios matched the reasoned prediction exactly. The total reduction (65.0%) fell inside the predicted 55 – 66% band, near its top because primes dominate the work. The running time fell by 65%, in line with the prediction that the remainder instruction dominates.

Size of the improvement and new costs. A factor of about 2.9 fewer divisions overall and exactly 3.0 for large primes, at no memory cost. The costs are a longer routine, a second test per pass (which can examine one candidate slightly beyond √n), a fixed price of two divisions for the tiny primes 5 and 7, and a higher risk of off-by-two errors that must be caught by validation against the original.

Where the original may still be preferable. (1) When readability and verifiability matter more than a constant factor: the original is four lines and is easy to explain in a viva. (2) For a handful of tickets (a single check of 1000003 costs 3.7 µs in the original; the saving is not noticeable). (3) For tiny n (5 and 7) the original is one division cheaper. (4) When the workload is mostly even numbers, both stop at the first division and are identical (n = 100: 1 and 1).

Limits of the redesign and what would be next. The complexity class is still O(√n); a wheel on 2, 3 and 5 would test 8 of every 30 numbers (theoretical 267 divisions for 1000003 – predicted only, not implemented), with diminishing returns for more code. When many tickets are checked, the Q5 sieve (O(N log log N) once, O(1) per query, O(N) memory) beats any trial-division variant: the measured break-even is about 31,300 random queries. The final recommendation is therefore: **use R2 (6k±1) for on-demand checks of individual tickets; build the sieve if the organisers verify the whole range or tens of thousands of tickets.**

Appendix A: source code and raw output (gcc 13.3.0)

q7_redesign.c

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

unsigned long checks;

/* ORIGINAL (Q5): trial division by every i with i <= n / i */
int prime_original(unsigned long n){
    checks = 0; if (n < 2) return 0;
    for (unsigned long i = 2; i <= n / i; i++) { checks++; if (n % i == 0) return 0; }
    return 1;
}
/* REDESIGN R1: test 2, then odd divisors only (the "improved" version of the Q5 screenshot) */
int prime_odd(unsigned long n){
    checks = 0; if (n < 2) return 0; if (n < 4) return 1;
    checks++; if (n % 2 == 0) return 0;
    for (unsigned long i = 3; i <= n / i; i += 2) { checks++; if (n % i == 0) return 0; }
    return 1;
}
/* REDESIGN R2: test 2 and 3, then only i = 6k-1 and 6k+1 */
int prime_6k(unsigned long n){
    checks = 0; if (n < 2) return 0; if (n < 4) return 1;
    checks++; if (n % 2 == 0) return 0;
    checks++; if (n % 3 == 0) return 0;
    for (unsigned long i = 5; i <= n / i; i += 6) {
        checks++; if (n % i == 0)       return 0;   /* 6k - 1 */
        checks++; if (n % (i + 2) == 0) return 0;   /* 6k + 1 */
    }
    return 1;
}
static double now(void){ struct timespec t; clock_gettime(CLOCK_MONOTONIC,&t); return t.tv_sec + t.tv_nsec*1e-9;
     }

int main(void){
    const unsigned long N = 1000003;
    unsigned long t[] = {97, 100, 7919, 999983, 1000001, 1000003, 2, 3, 25, 49, 35};
    int nt = sizeof t / sizeof t[0];
    printf("== Same inputs, divisions performed ==\n");
    printf("%-9s %-10s %9s %9s %9s %9s\n","n","result","original","R1 odd","R2 6k+-1","R2/orig");
    for (int k = 0; k < nt; k++){
        unsigned long n = t[k];
        int a = prime_original(n); unsigned long ca = checks;
        int b = prime_odd(n);      unsigned long cb = checks;
        int c = prime_6k(n);       unsigned long cc = checks;
        printf("%-9lu %-10s %9lu %9lu %9lu %8.2f %s\n", n, a?"prime":"composite", ca, cb, cc,
               ca ? (double)cc/ca : 0.0, (a==b && b==c) ? "" : "MISMATCH");
    }

    /* full validation + totals, split by prime / composite */
    unsigned long tot[3] = {0,0,0}, totP[3] = {0,0,0}, totC[3] = {0,0,0}, bad = 0, np = 0;
    unsigned long worst[3] = {0,0,0};
    for (unsigned long n = 2; n <= N; n++){
        int r0 = prime_original(n); unsigned long c0 = checks;
        int r1 = prime_odd(n);      unsigned long c1 = checks;
        int r2 = prime_6k(n);       unsigned long c2 = checks;
        if (r0 != r1 || r0 != r2) bad++;
        tot[0]+=c0; tot[1]+=c1; tot[2]+=c2;
        if (r0){ np++; totP[0]+=c0; totP[1]+=c1; totP[2]+=c2; } else { totC[0]+=c0; totC[1]+=c1; totC[2]+=c2; }
        if (c0>worst[0]) worst[0]=c0;
        if (c1>worst[1]) worst[1]=c1;
        if (c2>worst[2]) worst[2]=c2;
    }
    printf("\n== Every n = 2..%lu (mismatches between versions: %lu; primes found: %lu) ==\n", N, bad, np);
    const char *nm[3] = {"original","R1 odd","R2 6k+-1"};
    printf("%-10s %14s %14s %14s %10s\n","version","all n","primes only","composites","worst case");
    for (int v = 0; v < 3; v++) printf("%-10s %14lu %14lu %14lu %10lu\n", nm[v], tot[v], totP[v], totC[v],
        worst[v]);
    for (int v = 1; v < 3; v++)
        printf("%s vs original: all n = %.1f%% fewer, primes only = %.1f%% fewer, composites = %.1f%% fewer\n",
            nm[v],
               100.0*(1.0-(double)tot[v]/tot[0]), 100.0*(1.0-(double)totP[v]/totP[0]),
                   100.0*(1.0-(double)totC[v]/totC[0]));

    /* timing: all n up to N, best of 5 */
    printf("\n== Timing for n = 2..%lu (best of 5, -O2) ==\n", N);
    for (int v = 0; v < 3; v++){
        double best = 1e9; unsigned long sink = 0;
        for (int rep = 0; rep < 5; rep++){
            double t0 = now(); unsigned long cnt = 0;
            for (unsigned long n = 2; n <= N; n++)
```

```
                cnt += (v==0) ? prime_original(n) : (v==1) ? prime_odd(n) : prime_6k(n);
            double dt = now() - t0; if (dt < best) best = dt; sink = cnt;
        }
        printf("%-10s %.4f s (primes counted = %lu)\n", nm[v], best, sink);
    }
    /* timing: large primes only (worst case) */
    printf("\n== Timing: 20000 repeated checks of the prime 1000003 ==\n");
    for (int v = 0; v < 3; v++){
        double t0 = now(); unsigned long cnt = 0;
        for (int rep = 0; rep < 20000; rep++)
            cnt += (v==0) ? prime_original(1000003) : (v==1) ? prime_odd(1000003) : prime_6k(1000003);
        printf("%-10s %.4f s (sink %lu)\n", nm[v], now()-t0, cnt);
    }
    /* repeated work: how often small divisors are retried */
    unsigned long by2=0, by3=0, byAny=0;
    /* in original: number of divisions with i even, or i multiple of 3, across all primes */
    for (unsigned long n = 2; n <= N; n++){
        int p = 1; for (unsigned long i = 2; i <= n / i; i++){ if (n % i == 0){ p = 0; break; }
            if (i % 2 == 0) by2++; else if (i % 3 == 0) by3++; byAny++; }
        (void)p;
    }
    printf("\nOriginal, divisions that did NOT end the loop: total=%lu, with i even=%lu (%.1f%%), with i odd
        multiple of 3=%lu (%.1f%%)\n",
           byAny, by2, 100.0*by2/byAny, by3, 100.0*by3/byAny);
    return 0;
}
```

Raw output of q7_redesign

```
== Same inputs, divisions performed ==
n         result      original    R1 odd  R2 6k+-1   R2/orig
97        prime              8         5         4     0.50
100       composite          1         1         1     1.00
7919      prime             87        44        30     0.34
999983    prime            998       500       334     0.33
1000001   composite        100        51        35     0.35
1000003   prime            999       500       334     0.33
2         prime              0         0         0     0.00
3         prime              0         0         0     0.00
25        composite          4         3         3     0.75
49        composite          6         4         4     0.67
35        composite          4         3         3     0.75

== Every n = 2..1000003 (mismatches between versions: 0; primes found: 78499) ==
version             all n    primes only     composites worst case
original         67741504       50659938       17081566        999
R1 odd           34601109       25388823        9212286        500
R2 6k+-1         23739568       17030426        6709142        334
R1 odd vs original: all n = 48.9% fewer, primes only = 49.9% fewer, composites = 46.1% fewer
R2 6k+-1 vs original: all n = 65.0% fewer, primes only = 66.4% fewer, composites = 60.7% fewer

== Timing for n = 2..1000003 (best of 5, -O2) ==
original   0.2442 s (primes counted = 78499)
R1 odd     0.1268 s (primes counted = 78499)
R2 6k+-1   0.0852 s (primes counted = 78499)

== Timing: 20000 repeated checks of the prime 1000003 ==
original   0.0734 s (sink 20000)
R1 odd     0.0376 s (sink 20000)
R2 6k+-1   0.0240 s (sink 20000)

Original, divisions that did NOT end the loop: total=66820001, with i even=33640395 (50.3%), with i odd multiple
     of 3=11220986 (16.8%)
```

q7_worse_check.c

```
#include <stdio.h>
unsigned long checks;
int po(unsigned long n){checks=0; if(n<2)return 0; for(unsigned long i=2;i<=n/i;i++){checks++; if(n%i==0)return
    0;} return 1;}
int p6(unsigned long n){checks=0; if(n<2)return 0; if(n<4)return 1; checks++; if(n%2==0)return 0; checks++;
    if(n%3==0)return 0;
  for(unsigned long i=5;i<=n/i;i+=6){checks++; if(n%i==0)return 0; checks++; if(n%(i+2)==0)return 0;} return 1;}
int main(void){ unsigned long N=1000003, worse=0, same=0, better=0; long maxextra=0; unsigned long first[20];
    int nf=0;
 for(unsigned long n=2;n<=N;n++){ po(n); unsigned long a=checks; p6(n); unsigned long b=checks;
  if(b>a){worse++; if(nf<20) first[nf++]=n; if((long)(b-a)>maxextra) maxextra=b-a;} else if(b==a) same++; else
      better++; }
 printf("R2 better on %lu values, equal on %lu, worse on %lu (max extra divisions %ld)\nworse at n =", better,
     same, worse, maxextra);
 for(int k=0;k<nf;k++) printf(" %lu", first[k]); printf("\n"); return 0; }
```